KnoBAB

Making Logic Fast

Giacomo Bergami

Newcastle University, England, United Kingdom

LT_f@AAAI-SSS, 29th of March, 2023

Who Are We?

Assistant Prof. (Lecturer)

Dr Giacomo Bergami
[ʤaːkomo beːrgami]

PhD Student

Mr Samuel Appleby

Full Professor

Prof Graham Morgan

References

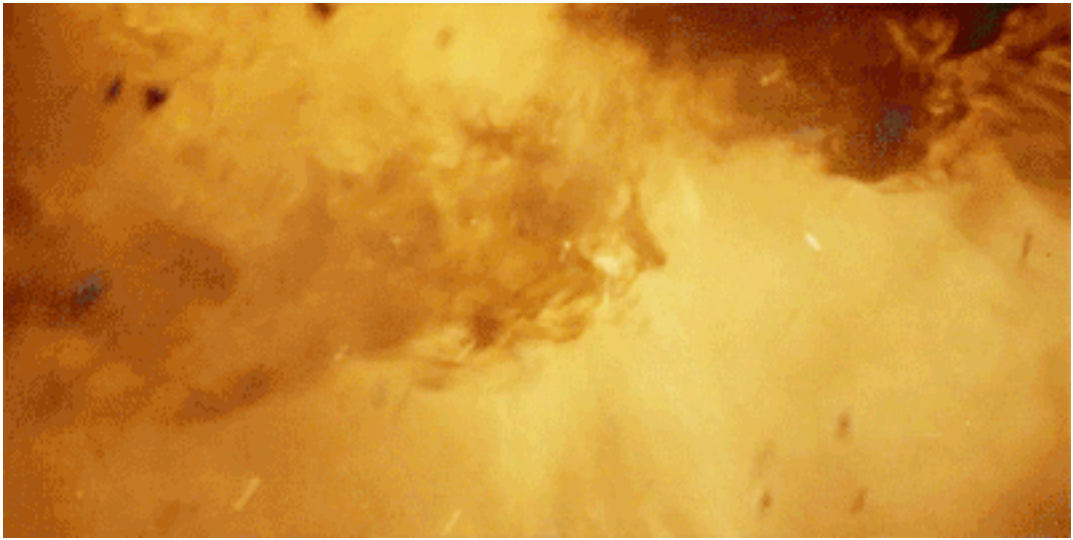

Samuel Appleby, Giacomo Bergami, and Graham Morgan.
Running temporal logical queries on the relational model.
In *IDEAS'22*, pages 134–143. ACM, 2022.


Samuel Appleby, Giacomo Bergami, and Graham Morgan.
Quickening data-aware conformance checking through temporal algebras.
In *Information (Switzerland)*, volume 14, page 60, 2023.


Samuel Appleby, Giacomo Bergami, and Graham Morgan.
Enhancing declarative temporal model mining in relational databases: A preliminary study.
In *IDEAS'23 [In Press]*, 2023.

And now. . . (1971)

And now. . . (1971)



What is KnoBAB? (1/2)

- Bridging the gap between theoretical research and practical use case scenarios.
- To the best of our knowledge, current approaches in the field have at least one of the following shortcomings:
  1. Do not reap big data algorithms for carrying out computations efficiently
  2. Most of the time, the users are forced to "reduce" a data-aware domain to a dataless one to carry out business process mining operations (e.g., trace repairs or alignments).
  3. KnoBAB shows that it is possible to import a lot of good practices from database field to the benefit of a larger community.
  4. Either assume one fixed declarative language (*Process Query Language*), or are mainly GUI-driven (*ProM*, *RuM*).

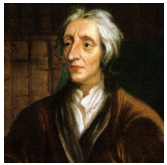
What is KnoBAB? (2/2)

In order to perform optimizations, we performed the following assumptions:

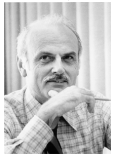
- We are not interested in empty traces, as they have no events!
- Traces' payloads can be represented as a distinctive event (`__trace_payload`) appearing at the beginning of the trace.
- Differently from the current standard, keys are not duck typed, but are associated to only one specific data-type!
- Boolean and Integer representation are mapped into double precision floating points.
- For each database, we consider a minimum and maximum possible string value length.

What is xtLTL$_f$?

- Despite LTL$_f$ describes a way to calculate the satisfiability of a trace starting from its beginning and providing temporal quantification on specific events, any relational algebra works in the opposite way, from the data being accessed.
  - $\Rightarrow$ next ($\bigcirc\varphi$, $X\varphi$) must be evaluated from φ towards the preceding event, if any.
- Walking on the footsteps of Declare, we might assume to have activation and target condition to be tested. Differently from LTL$_f$, we also need to explicitly express data correlation conditions.

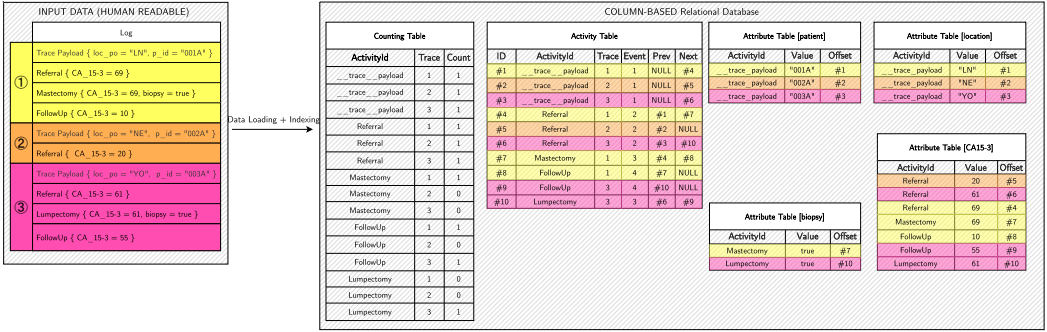


"Logic is the anatomy of thought"
John Locke (1632-1704)



Edgar Codd (1923-2003)

I. Column-Store Data Representation



Column-Base storage work under the assumption that it is always possible to decompose a relation $\Re(\texttt{id}, A_1, \ldots, A_n)$ into n tables $\Re_i(\texttt{id}, A_i)$ such that $\bowtie_{1 \leq i \leq n} \Re_i = \Re$

For the result representation, we relax the 1NF for representing the result, an ordered sequence of nested records $\langle i, j, L \rangle$ entailing that the j-th event of the i-th trace satisfies the conditions of ρ. L lists all the data activation, target, and Θ correlation conditions being witnessed.

II. Temporal Algebraic Operators ($\texttt{xtLTL}_\text{f}$)

$\texttt{xtLTL}_\text{f}$: Syntax

Given optional A/T fields, an $\texttt{xtLTL}_\text{f}$ expression is defined as follows:

$$\rho ::= \mu \mid \nu \mid \beta \mid \delta$$

$$\mu ::= \mathsf{Activity}^{\mathcal{L},\tau}_{A/T}(\mathsf{a}) \mid \mathsf{Compound}^{\mathcal{L},\tau}_{A/T}(\mathsf{a}, \mathit{lower} \leq \kappa \leq \mathit{upper}) \mid \mathsf{First}^{\mathcal{L},\tau}_{A} \mid \mathsf{Last}^{\mathcal{L},\tau}_{A} \mid \mathsf{Init}^{\mathcal{L}}(\mathsf{a}) \mid \mathsf{Ends}^{\mathcal{L}}(\mathsf{a})$$

$$\nu ::= \mathsf{Exists}_n(\rho) \mid \mathsf{Absence}_n(\rho) \mid \mathsf{Init}(\rho) \mid \mathsf{Ends}(\rho) \mid \mathsf{Next}^{\tau}(\rho) \mid \mathsf{Globally}^{\tau?}(\rho) \mid \mathsf{Future}^{\tau?}(\rho) \mid \mathsf{Not}^{\tau?}(\rho)$$

$$\beta ::= \mathsf{Until}^{\tau?}_{\Theta}(\rho_1, \rho_2) \mid \mathsf{And}^{\tau?}_{\Theta}(\rho_1, \rho_2) \mid \mathsf{Or}^{\tau?}_{\Theta}(\rho_1, \rho_2)$$

$$\delta ::= \mathsf{AndFuture}^{\tau}_{\Theta}(\rho_1, \rho_2) \mid \mathsf{AndGlobally}^{\tau}_{\Theta}(\rho_1, \rho_2) \mid \mathsf{AndNextGlobally}^{\tau}_{\Theta}(\rho_1, \rho_2)$$

The preliminary results show that $\texttt{xtLTL}_\text{f}$ is at least as expressive as LTL_f where the distinction between timed and untimed operators perserve the expected "good" properties. We also provided a formalisation of activation, target, and correlation condition for data-aware scenarios.

II. Temporal Algebraic Operators ($\texttt{xtLTL}_\texttt{f}$)

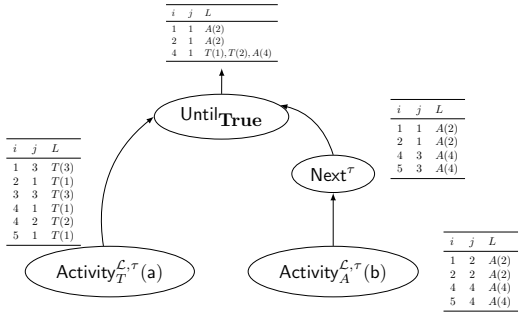
Tutorial

Let's suppose to evaluate $\mathsf{a}\mathcal{U}\bigcirc(\mathsf{b})$ over the following log:

$$\mathcal{L} = \{\mathsf{dbacd}, \mathsf{ab}, \mathsf{cda}, \mathsf{aadb}, \mathsf{addb}\}$$

This can be expressed as

$$\mathsf{Until}_{\mathbf{True}}(\mathsf{Activity}_T^{\mathcal{L},\tau}(\mathsf{a}), \mathsf{Next}^{\tau}(\mathsf{Activity}_A^{\mathcal{L},\tau}(\mathsf{b})))$$



III. Conjunctive and Aggregation Queries

Without the need of changing the operators' semantics nor the results being returned, the same operators can express over the same "model" all of the following queries, by simply changing the root node of the query plan:

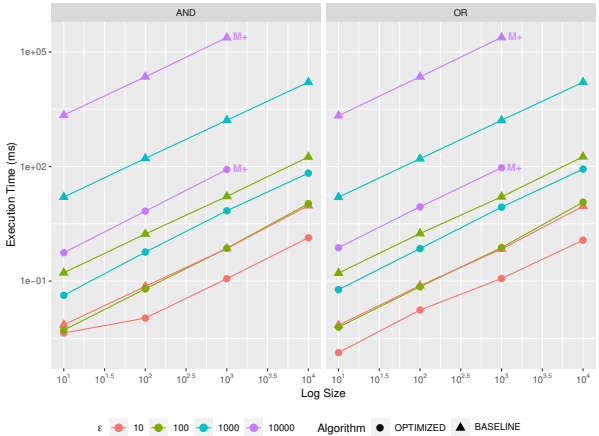
$$\textsc{ConjunctiveQuery}(\rho_1, \ldots, \rho_n) = \mathsf{And}_{\mathbf{True}}(\rho_1, \ldots, \mathsf{And}_{\mathbf{True}}(\rho_{n-1}, \rho_n))$$

$$\text{Max-SAT}(\rho_1, \ldots, \rho_n) = \left(\frac{|\{l | \exists j, L.\ \langle i, j, L \rangle \in \rho_l\}|}{|\mathcal{M}|} \right)_{\sigma^i \in \mathcal{L}}$$

$$\textsc{Confidence}(\rho_1, \ldots, \rho_n) = \left(\frac{|\{i | \exists j, L.\ \langle i, j, L \rangle \in \rho_l\}|}{|\mathsf{ActLeaves}(\rho_l)|} \right)_{c_l \in \mathcal{M}}$$

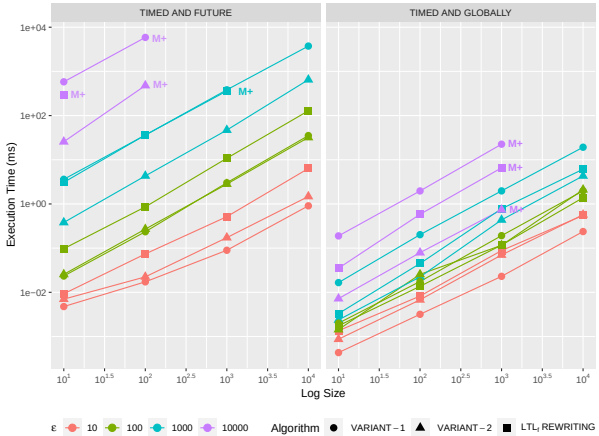
$$\textsc{Support}(\rho_1, \ldots, \rho_n) = \left(\frac{|\{i | \exists j, L.\ \langle i, j, L \rangle \in \rho_l\}|}{|\mathcal{L}|} \right)_{c_l \in \mathcal{M}}$$

IV. Query Plan Optimisation (1/4)



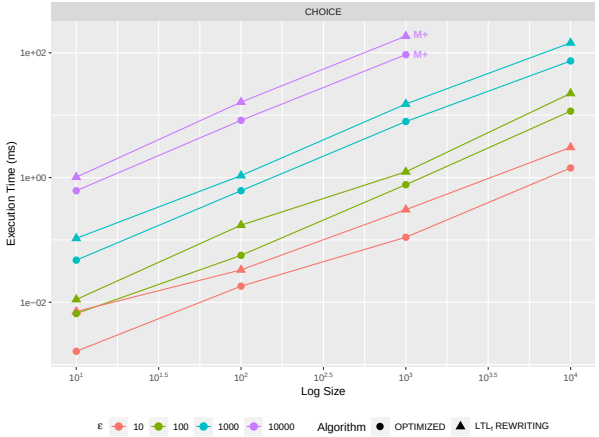
1. For untimed And and Or, we can provide different implementations, one being more memory efficient (Out of Primary Memory) and faster than the other.

IV. Query Plan Optimisation (2/4)



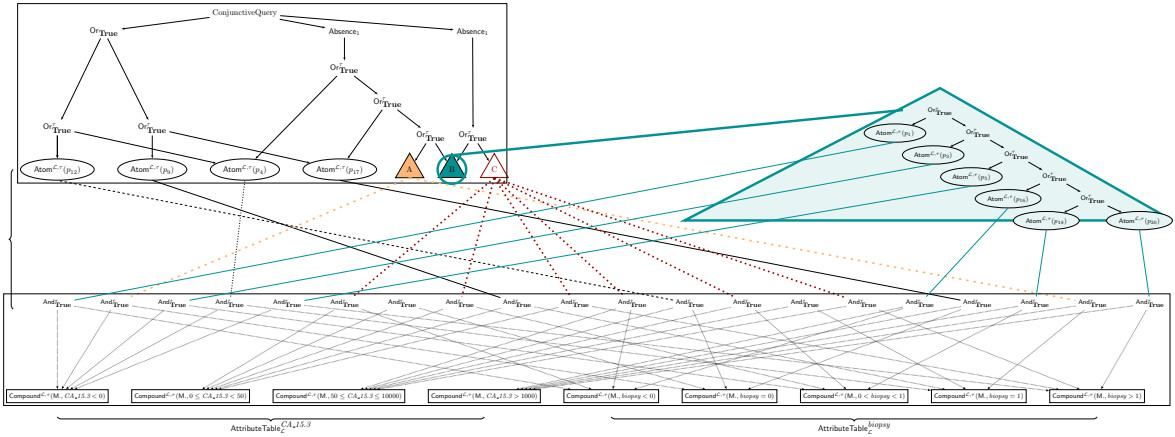
2a. As customary in the database world, by defining *ad hoc* operators subsuming the evaluation of recurrent sub-expressions, we can design better algorithms!

IV. Query Plan Optimisation (3/4)

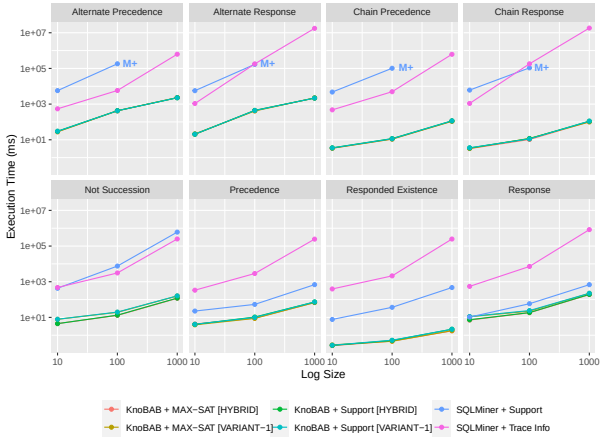


2b. Given that our operators admit $\mathsf{Or}_\Theta(\mathsf{Future}(\rho_1), \mathsf{Future})(\rho_2)) = \mathsf{Or}_\Theta(\rho_1, \rho_2)$ without breaking the "correctness", we can reduce the amount of unnecessary operations as well as memory allocations.

IV. Query Plan Optimisation (4/4)

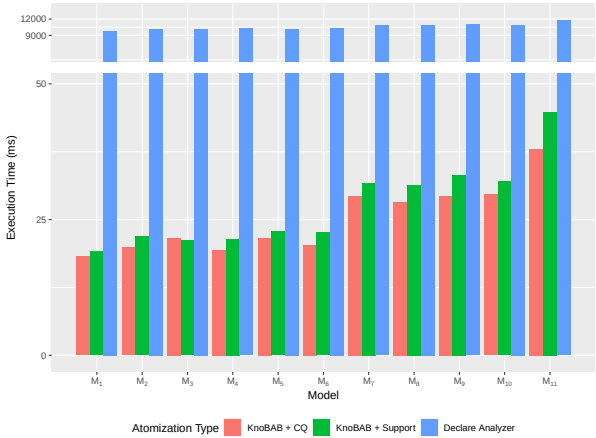


V. Temporal Model Mining...



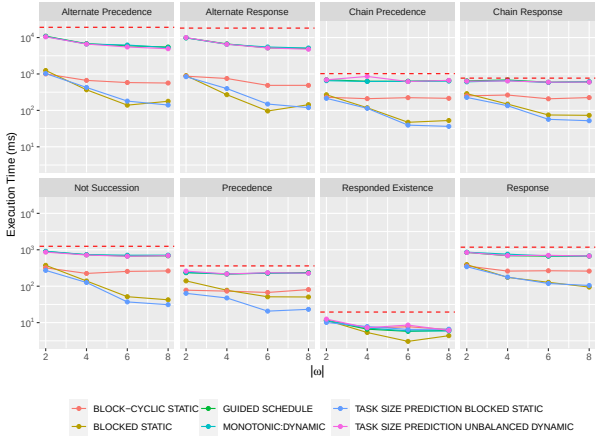
SQLMiner. If compared to other SQL-driven mining algorithms (where a given set of declarative clauses to be tested is previously generated through *propositionalisation*), our results show that our solution outperforms similar querying tasks on PostgreSQL.

V. . . . and Conformance Checking



Declare Analyzer. Our solution also outperforms ad-hoc Java implementations for computing data-aware conformance checking solutions.

VI. Query Plan Parallelisation



The DAG query plan allows running a *topological sort*. By "*layering*" the query plan, we can schedule which operators might be parallelised. Different scheduling policies give different speed-ups exploited by reducing page faults!

VII.-VIII. Customisability/Interoperability

```
load TAB "/home/giacomo/test.tab" as "tab";                    /* Loading Log */

display ACTTABLE for "tab";                                    /* Dump the internal representation as CSV */

auto-timed queryplan "aaai23" {                                /* Setting the semantics for declarative op. */
 template "Example" args 2 := (EXISTS  1 t #1 activation) U
                              (NEXT EXISTS  1 t #2 target)
};

model-check                                                    /* Conformance Checking */
        declare "Example" ("a", true, "b", true)               /* Manually providing the model */
using "TraceIntersection" over "tab"                           /* Conjunctive query + Log name */
plan "aaai23" with operators "Hybrid"                          /* Semantics to adopt, type of algorithms */
display query-plan;                                            /* Provide the DAG query plan */
```

- By supporting $\texttt{xtLTL}_\text{f}$ instead of declarative clauses, our system allows the definition of customary templates (`queryplan`s) that can be loaded at runtime.
- The running example could be run in KnoBAB (`knobab_server`) with the commands given above on a console (`knobab_client`).

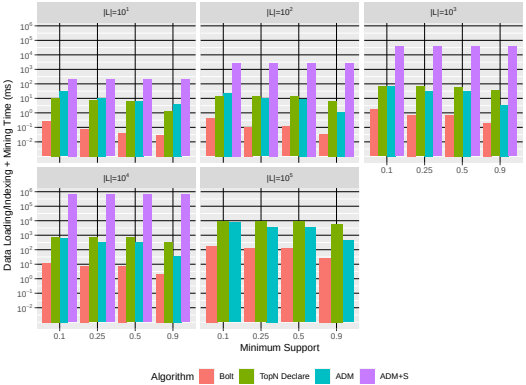
VII.-VIII. Customisability/Interoperability

```
import pandas as pd
from io import StringIO

# Setting up the client
cli = uk_knobab_kinj_KINJ("localhost", 8795)
# Generating a loading data query
q = uk_knobab_kinj_QueryCompiler.LoadDataQuery(uk_knobab_kinj_LogFormat.TAB,
                                               "/home/giacomo/test.tab", "tab",
                                               False,False,False)
# Sending the data loading request to the server
cli.PerformQuery(q)
# Asking to dump the ActivityTable as a CSV string
q = uk_knobab_kinj_QueryCompiler.DisplayData(uk_knobab_kinj_Displays.Display(uk_knobab_kinj_DisplayTable.ACT_TABLE,"tab"))
result = cli.PerformQuery(q)
# Loading the CSV as a Pandas Dataframe
pd.read_csv(StringIO(result.message))
```

- KnoBAB can be queried as a restful HTTP server (`knobab_server`) and has API (**KINJ**) supporting major programming language by transpiling code in Haxe!
- The above example shows how to perform a query and getting answers in Python by connecting to the KnoBAB server.

IX. Minimal Model Description Length

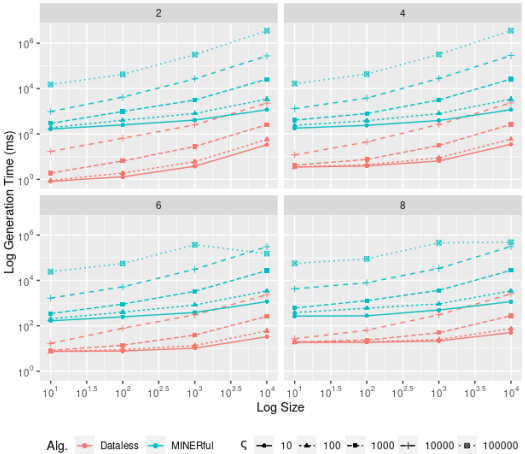


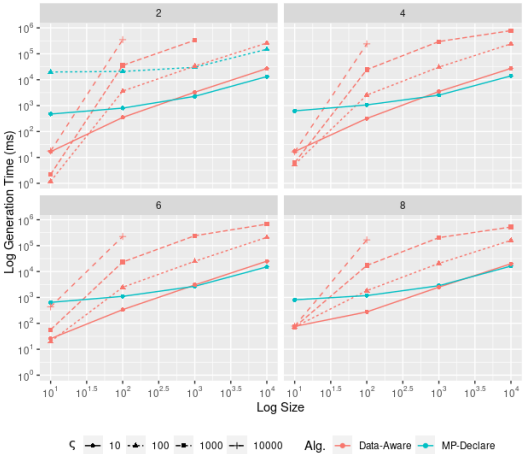
- It is possible to use KnoBAB as a library for re-implementing existing algorithms (**ADM**).

Log Gen. Model, $\mathcal{M}^+$	**Bolt**	**TopN**	**ADM**	**ADM+S**
ChainPrecedence(b, c)	✓	✗	✓(+Choice,CoExist.)	✓(+Choice,CoExist.)
ChainResponse(d, e)	✓	✗	✓(+Choice,CoExist.)	✓(+Choice,CoExist.)
Choice(f, g)	RespExistence(g, f)	✓(+Choice(g, f))	✓(+Choice(g, f))	✓(+Choice(g, f))
ExclChoice(h, a)	✓	✗	✗	✓(+ ExclChoice(a, h))
Exists(b, 1)	✓	✗	✓	✓
Init(e)	✓	✓	✓	✓
Precedence(c, b)	✓	✗	✓(+Choice,RespEx.,CoEx.)	✓(+Choice,RespEx.)
RespExistence(e, f)	Response(e, f)	✓	✓	✓
RespExistence(h, i)	CoExistence(h, i)	✗	✓(+ CoExistence(h, i))	✓
RespExistence(i, h)		✗	✓(+ CoExistence(i, h))	✓
Response(e, f)	✓	✓(+ RespExistence(e, f))	✓(+ RespExistence(e, f))	✓(+ RespExistence(e, f))
$\lvert\mathcal{M}_d\rvert$	197	112	548	684
$\lvert\mathcal{M}_{\geq 0.999}\rvert$	46	35	119	127
Mining time (μ)	$1.73 \cdot 10^6$ms	$5.17 \cdot 10^5$ms	$1.40 \cdot 10^6$ms	$3.29 \cdot 10^5$ms

- Our latest work shows that it is possible to obtain compact temporal models without additional learning tasks or "pruning scores" by only enforcing non-zero confidence and traversing the lattice of the declarative patterns.
- Our solution also outperforms previous solutions and renditions of previous algorithms on KnoBAB.

Extra: Synthetic Logs via Graph Algorithms [Under Review]





X. Transparency and Result Replicability

- The codebase is associated to test sets run with googletest.
- The papers' datasets are released on the Open Science Framework.
- We exploit GitHub Workflows for guaranteeing that the released code compiles and that all the tests are run satisfactorily.

`https://github.com/datagram-db/knobab`